

Error assessment and code speed-up for SOLPS-ITER

Developed within the framework of WP-CD 2016

Manual version 4.2

M. Baelmans¹, P. Börner², M. Blommaert¹, K. Ghoo¹, W. Dekeyser¹, and S. Carli¹

¹KU Leuven, Department of Mechanical Engineering, 3001 Leuven, Belgium

²Institute of Energy and Climate Research (IEK-4), FZ Jülich GmbH, D-52425 Jülich, Germany

September 5th, 2019

Executive Summary

In 2016 the work on SOLPS code speed-up in terms of accuracy is finalized by its implementation in the SOLPS-ITER version. The procedure essentially consists of three subsequent steps. Every step provides an additional level of information on simulation results, its related errors and optimal use of numerical parameters. The first step is the reference user run. In this step, the averaging procedure is used to achieve simulation results with reduced statistical noise. Therefore, it is recommended to be used in all cases. In addition, variances are monitored to estimate the statistical error. It is shown in [1] that this averaging procedure introduces a small bias. In this first step, however, the value of this bias is not yet known. Nevertheless, it becomes already apparent from residual evaluation that increased accuracy is achieved.

In order to achieve optimal numerical parameters given a required accuracy of simulation results for a specific case and its related grid size, the bias needs to be assessed. To this end, runs with a different number of Monte Carlo particles are needed. Based on theoretical error reduction rates [1, 2] for statistical error and bias, the optimal numerical parameters can be set to minimize CPU for a desired accuracy. The averaging procedure enables us to drastically reduce the statistical error in the solution within a reasonable computational time. As a result of this, we now can go even one step further and assess the discretization error, i.e. the error related to the finite grid size.

The procedure as described in this manual is applied to a slab test case first and subsequently tested for a Deuterium only ITER F12 case, a C-MOD Deuterium only case and a C-MOD case with Deuterium and Boron.

This report provides a practical user manual for the averaging procedure to be used to determine numerical parameters for code speed-up. Information on underlying theoretical and numerical studies can be found in [1–3].

1 Introduction

This report focuses on providing the necessary documentation to serve its users. In the following section the basic run with averaging procedure is documented. It is implemented with the intention for general use, meaning for use by all SOLPS-ITER users.

The expert level to assess the errors and to speed-up the code can be performed now following the methodology described in [3]. By assessing the different contributions an optimal set of numerical

parameters (# Monte Carlo particles and # of iterations) can be obtained. It is recommended to do this for each new case or whenever strong changes in plasma parameters occur. The procedure allows to obtain accurate results in a much faster way. This opens perspective to evaluate discretization errors. This first version of the manual focuses on general user information to enable averaging. This manual will be further expanded in the future based on the user's experience and needs.

2 Running SOLPS-ITER with averaging procedure

2.1 Introduction

The averaging procedure is the essential first step to improve the code results and to assess the statistical error. Essentially nothing changes with respect to the iteration process, the run, itself. The tool provides an improved interpretation of the simulation results via averaging over iterations only. This iteration averaged value is not fed back to the iteration procedure. From a theoretical point of view, it is shown [1] that an averaging procedure over successive iteration steps of a coupled Finite Volume (FV) / Monte Carlo (MC) results in a solution with smaller **statistical error** than the one associated with the instantaneous plasma solution. By averaging, the (usually small) **finite particle bias** becomes important and needs to be taken into account. The latter is a deterministic error and therefore called a bias.

In the following we will distinguish between an instantaneous plasma solution and an iteration averaged solution.

- The **instantaneous plasma solution** is the solution at some point in the iteration process; it is this value that thus far always was considered as the solution of the set of governing equations.
- The **iteration averaged solution** is computed by averaging instantaneous plasma solutions achieved over subsequent iterations.

Former studies revealed that the statistical error of iteration averaged solutions decreases inverse proportional to the amount of iterations I over which is averaged ($\propto 1/\sqrt{I}$). Similarly, the error also decreases with increasing number of MC particles P ($\propto 1/\sqrt{P}$). On the other hand the bias introduced through averaging is observed to be smaller than the statistical error of the instantaneous field for all applications studied thus far and decreases inverse proportional with P .

It is clear that the former statements for statistical errors hold in case the plasma fields used for averaging are in statistically steady state. Therefore we distinguish between an initial phase and the statistically steady state phase.

- **Statistically steady state** means that the iteration averaged value is no longer changing over iterations.
- In contrast, during the **initial phase**, iteration averaged solutions clearly show a transient behaviour

In order to limit disk space needed for averaging, we distinguish between data needed for monitoring and data needed for establishing accurate results and related estimation of the statistical error. For monitoring purposes there is a need now for determining the end of the initial phase. This is provided by batch averages of a limited set of properties which are written to the b2batch.nc file. More specifically we distinguish batch averages from running averages.

- **Batch averaging** means that an average is taken over a given number of iterations. When the number of iterations is reached a next batch average is calculated over the subsequent number of iterations.

- In contrast, we will refer to **running averages** when averaging, once started, is continued over all subsequent iterations, even in case a run is restarted.

Running averages are used once the initial phase is over and statistically steady state is reached. At this instance the procedure starts with providing running averages of all plasma properties and source terms. To enable continuation of the averaging procedure even after restarting the run, the plasma file with running average values and running average sources, as well as their variances, will be stored. Thus, it is guaranteed that the averaging procedure can be continued. Again along the run, it is interesting to observe how well the averaged values are obeying the governing equations. This is assessed by monitoring the residuals of the averaged plasma solutions. To this end, running averages of plasma profiles and sources are saved at intermediate iteration numbers to keep track of the convergence. The different aspects in the averaging procedure are illustrated in figure 1.

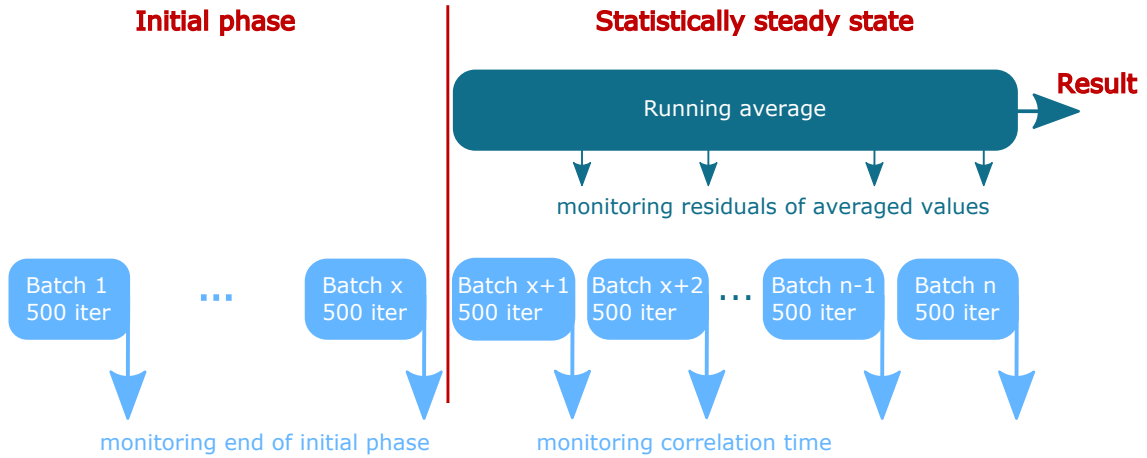


Figure 1: Schematic of the averaging procedure

In the following the procedure to practical use of iteration averaging is described. An overview of the new variables introduced for the averaging procedure can be found in section 2.6.

2.2 Step 1a: Batch averaging in the initial phase

When starting a new test case, envisaging the averaging procedure, we first need to check when the initial phase is over. To this end we monitor batch averages of specific quantities. Only after this initial phase the solution reaches its statistically steady state and the averaging procedure to reduce the statistical noise can start. The quantities to be monitored in the initial phase are the same type of quantities as used for monitoring the instantaneous values. A subset of these quantities, now with its batch averaged values, is made available in the **b2batch.nc** file by default (i.e. when `b2mndt_av_batch_all` ≥ 0 , see also section 2.3). These batch averages are less noisy than the instantaneous ones and as such are better suited for visualising transient behaviour.

In addition to the regular input of your case add the input parameters as listed in table 1.

Note 1: The computation of batch averages of default quantities is not limited to the initial phase, (assuming `b2mndt_av_ntim_batch` > 0). These quantities are also used for monitoring and eventually statistical error assessment in the statistically steady state.

Note 2: It is possible to extend the predefined variables. This requires further implementation work (cfr. set of predefined variables for the regular `b2batch.nc` file). A full list of monitored instantaneous

Variable	Value	Description
in b2mn.dat		
b2mndt_av	0 (Def.)	No computation of the running average
b2mndt_av_ntim_batch	e.g. 500 (Def.)	When larger than 0, number of iterations used to compute batch averages
b2mndt_av_batch_all	0 (Def.)	Standard output for batch averages
in input.dat		
ninitl	<0	parameter to ensure full randomness of the seed to be done in block 7 for each stratum and in block 13 in 1 st line of integers 2 nd position

Table 1: Input parameter values for initial phase

quantities can be found in the SOLPS-ITER manual, Appendix E.

Note 3: For expert use, it is possible to set `b2mndt_av_batch_all = 1`. In this case, batch averages of 2D fields of the plasma states and their variances are provided in `./batch_av/batch_av.XXXX` every `b2mndt_av_ntim_batch` iterations. Be aware that this will consume a lot of disk space!

2.3 Step 1b: Determine transient phase

To determine the transient phase, batch averages of (selected) quantities are stored in `b2batch.nc` during the run (see step 1a). Determine with visual inspection whether the file is in its statistically steady state. When you reach this phase, you start the running average as described in step 1c. The following list of quantities are available for batch-monitoring:

Variable	Description
nesepm_av, nesepi_av, nesepa_av	separatrix electron density at outer midplane, inner and outer target
nemxip_av, nemxap_av	max electron density at inner and outer target
tesepm_av, tesepi_av, tesepa_av	separatrix electron temperature at outer midplane, inner and outer target
temxip_av, temxap_av	max electron temperature at inner and outer target
tisepm_av, tisepi_av, tisepa_av	separatrix ion temperature at outer midplane, inner and outer target
timxip_av, timxap_av	max ion temperature at inner and outer target
posepm_av, posepi_av, posepa_av	separatrix electric potential at outer midplane, inner and outer target
pomxip_av, pomxap_av	max electric potential at inner and outer target

Table 2: Default monitored plasma parameters

These quantities should be plotted against the batch time, which is stored in the array **batchsa** in the `b2batch.nc` file. **batchsa** is the total amount of simulated time (in seconds) at which the batch average is computed (the last iteration of the batch). Thus, the batch averages can easily be plotted on a same axis as the other time dependent quantities (`nesepm`, `nesepi`, etc.). The latter are plotted against the **timesa** array (also in seconds). The difference between **batchsa** and **timesa** is that **timesa** is defined at time steps where the standard time dependent quantities are written, while **batchsa** is defined at time steps where batch-averages are written.

To visualize the batch averaged quantities use *2dt_av nsepm_av nsepi_av* etc. The script is available in `$SOLPSTOP/scripts`. For the slab case the separatrix density values at the outer midplane, inner and outer target are shown in figure 2 (left), whereas the evolution of the maximum density at inner and outer target is shown in figure 2 (right). The instantaneous residuals obtained with the *resall_D* are shown for illustration in figure 3.

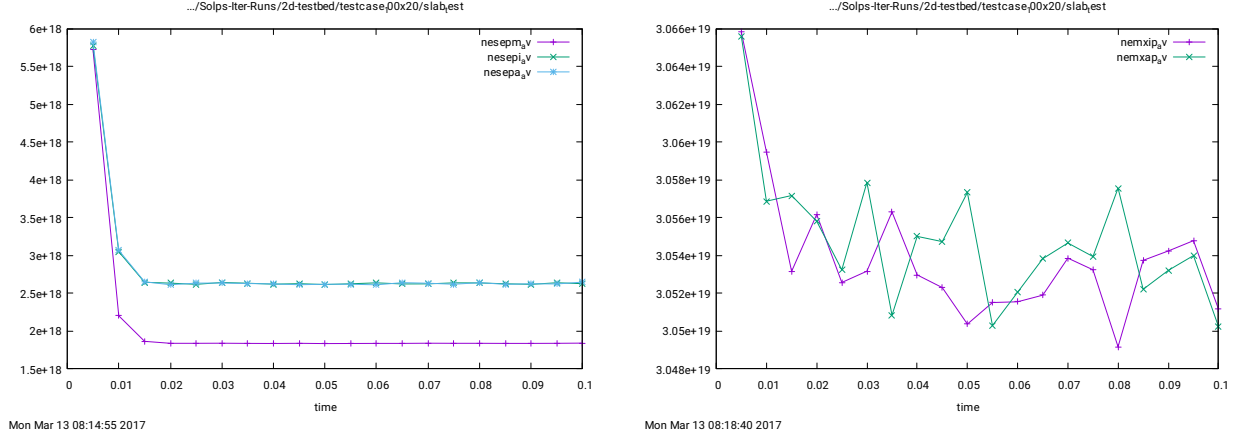


Figure 2: Sample of batch averaged quantities for a slab test case to monitor the end of the initial phase (20 batches of 500 it.): separatrix density values at the outer midplane, inner and outer target (left), maximum density at inner and outer target (right)

2.4 Step 1c: Compute averaged solution and its residuals

Once the initial phase is over, the effective running averaging procedure is started. During the averaging procedure in steady state, the averaged plasma state values, the averaged sources and their respective variances are saved into a file called **b2faver** at the end of the run. The averaging procedure starts from 0 in case no initial averaging file **b2faveri** is available. For monitoring the residuals of the running averaged profiles the simulation results are saved at regular times (controlled by **b2mndt_av_ntim_run**). This data is stored in `./run_av/run_av.XXXX` files.

The running averaging procedure is activated by adding the following input to your case:

Variable	Value	Description
b2mndt_av	1	Computation of the running average
b2mndt_av_continue	1 (Def.)	Continuously perform running averages
For monitoring purposes		
b2mndt_av_ntim_run	e.g. 1000 (Def.)	Number of it. for writing running averages
b2mndt_av_ntim_batch	e.g. 500 (Def.)	Number of it. for computing batch averages for monitoring purposes only
b2mndt_av_batch_all	0 or 1	Standard or expert output for batch averages

Table 3: Input parameter values in statistically steady phase

Once the `run_av` files are produced the residuals can be monitored by calling the script **res_av**. This script requires the user to have an **input_fin.eir** file present in the run directory, which is a copy of

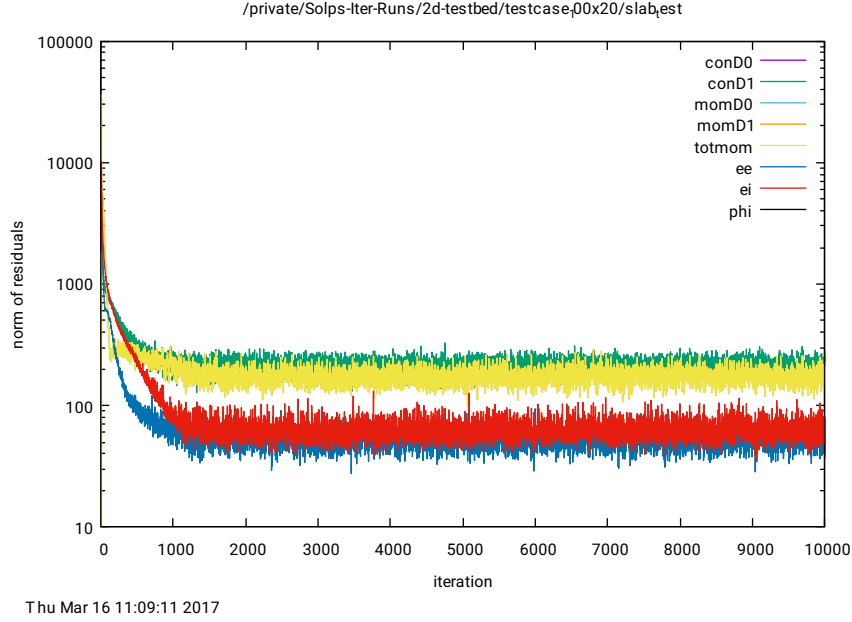


Figure 3: Instantaneous residuals in initial phase

input.dat with a large number of EIRENE particles. If not present, the script will look for this file in baserun. The script, upon successful running, will produce the resall_D plot. Furthermore, it will create a b2ftrace_av file which can be used for additional plots by renaming it to b2ftrace.

The final plasma simulation results can now be found in **b2fstate_average**. The EIRENE simulation results need to be obtained from this plasma as an initial state and using a high number of particles to reduce statistical noise. To this end the parameters listed in table 4 need to be set in the b2mn.dat file. A residual plot for the slab test case is shown in figure 4. The number of iterations here indicate the number of batches (in this case to be multiplied with $b2mndt_av_ntim_batch = 500$ to obtain the overall number of iterations during averaging). It should be noted that the residuals of the instantaneous plasma values (see figure 3) are very noisy and are several orders of magnitude larger than the ones obtained through averaging (figure 4). The averaging procedure already results in low residuals after the first batch and further decrease for some equations as the statistical error is further reduced. Continuation of the run leads to a further reduction of the electron energy residuals, meaning that the statistical error is further reduced by continuing the averaging procedure.

Further, when the stagnation level for the residuals is reached, it is now dominated by the bias and gives a first estimate of the bias error introduced by the finite amount of particles used in the EIRENE run. This stagnation level can be lowered by using more EIRENE particles. In this particular example it is clear that the residual for the energy equation is still sensitive to statistical noise rather than by the finite sampling bias, whereas continuity, momentum and ion energy reach their saturation by the bias at a much earlier stage.

Note 1: When setting $b2mndr_av_read = 1$, all the EIRENE sources within B2.5 are overwritten by the averaged sources. In order to obtain the EIRENE output (e.g. fluxes on walls) with the large number of particles, the user has to re-set $b2mndr_av_read = 0$, $b2mndt_rx = 0.0$, $b2mndr_ntim = 1$ and use as initial plasma the plasma state obtained by reading the averaged sources.

Note 2: The importance of starting the averaging procedure only after the initial phase is illustrated in figure 5. Here are plotted the residuals of the running averages where averaging was started right from

Variable	Description
in b2mn.dat	
b2mndr_av_read = 1	Read the initial plasma state from the averaged plasma b2faveri
b2mndt_av_continue = 1	Continue perform running averages
b2mndr_ntim = 2	Only two iteration steps are required to give the averaged profile to EIRENE
b2mndt_rxf = 0.0	No changes to the averaged plasma profiles are made
in input.dat	
block 7, 1st row of real numbers, 2nd position	Multiplicative factor to enhance simultaneously all the numbers for particles released at the different strata

Table 4: Input parameters to obtain the final plasma and neutral particle solution

the beginning, i.e. from the initial plasma solution obtained with a neutral fluid model. The residuals will finally also drop to a small value, the computational effort is however significantly increased!

Note 3: In order to guarantee also a backup file for the averaging procedure, running averages and variances are stored at regular times. This procedure is similar to the one foreseen for the instantaneous plasma parameters. The plasma data is stored in `plasmastate_aver.XXXX`.

Note 4: For the slab test case, the statistiscal error can easily be made small enough to enable symmetry checks for this symmetrical test case. Thus, this test case brings additional verification for new implementations in the coupling between B2.5 and EIRENE. As an example, comparison of density and ion temperature profiles obtained with B2.5-EIRENE (SOLPS-ITER) are shown in figures 6 and 7 respectively, for the symmetric test slab case. Relative differences between left and right target are in the order of 10^{-5} to 10^{-4} .

2.5 Step 1d: Assessment of the statistical error

At this stage the first error contribution, namely the statistical error, for a SOLPS-ITER averaged solution can be quantified starting from the batched averaged values stored in **b2batch.nc** for the last run executed. The error assessment takes account for the overall number of iterations used for averaging, eventually started in previous runs and continued in the last one. The statistical error for all variables stored in the `b2batch.nc` file is obtained by calling **b2ye**. The result is stored in **b2ye.prt**. To determine the statistical error the correlation time between batches is needed. To this end the batch information from **b2batch.nc** is used. This data is limited to the one provided in the last run. The longer this run the more reliable the error assessment will be. The two methods described in [4,5] are implemented to asses the correlation time between two batches. To be conservative the highest value for the correlation time is used for the statistical error.

To demonstrate the output, the averaging procedure was executed over 100 batches of 500 iterations each. The initial phase consisted of 100 batches of 500 iterations each. The correlation time ranges between 2 and 15 for these values. The averaged values and their relative statistical error are listed for this slab case in table 5. It should be noted that the maximum values are not shown in the table as they occur at the preset core boundary for this case and are therefore constant. It is clear that the statistical error can be further lowered by continuing the averaging procedure. It will further decrease

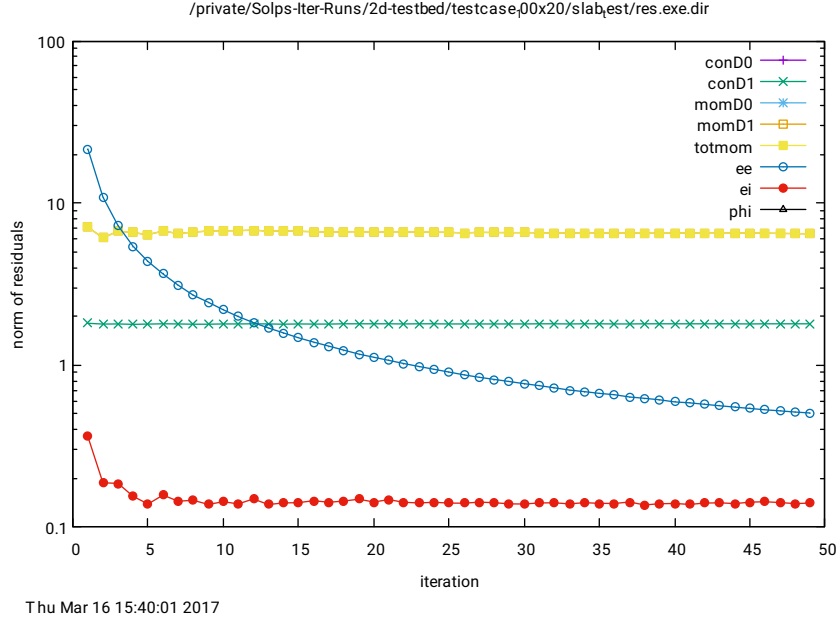


Figure 4: Residuals of the running averaged solution for a slab test case

inversely proportional with the number of iterations.

2.6 Description of the additional input variables introduced

The new variables introduced for the averaging procedure are tabulated in table 6.

3 Optimal parameters for code speed-up - expert level

In order to achieve optimal numerical parameters to minimize the sum of statistical error and finite particle bias, the user needs to derive the governing constants for the error reduction rates for a specific quantity of interest, e.g. the overall heat flux to the outer target. To this end the user needs to run the SOLPS-ITER case with three different values for the number of particles launched in EIRENE. These values are preferentially taken at least a factor 10 apart, but can be performed with a reduced amount of particles rather than with an increased number. Thus the bias is more easily provoked. Once the numerical constants (cfr. the ones listed in [3], table 3) are extracted from the **b2batch.nc** and the **b2favere** file, the optimal value can be computed with the Matlab-script *optimalP.m*. For further information the reader is referred to [3].

As a consequence of the related code speed-up, the code can now also handle finer grids within an acceptable time frame. Given the small statistical errors that can be achieved, the discretization error can be assessed following the conventional CFD approaches. The assessment of the grid discretization error is performed for a partially detached ITER case (#2011). Its results can be found in [4].

Note 1: If the number of particles launched in EIRENE is reduced, then parallel computation might not be efficient anymore due to the overhead in communication within different MPI processes. This is case-dependent as well as hardware-dependent and the user should assess which is the most efficient number of processes to be adopted.

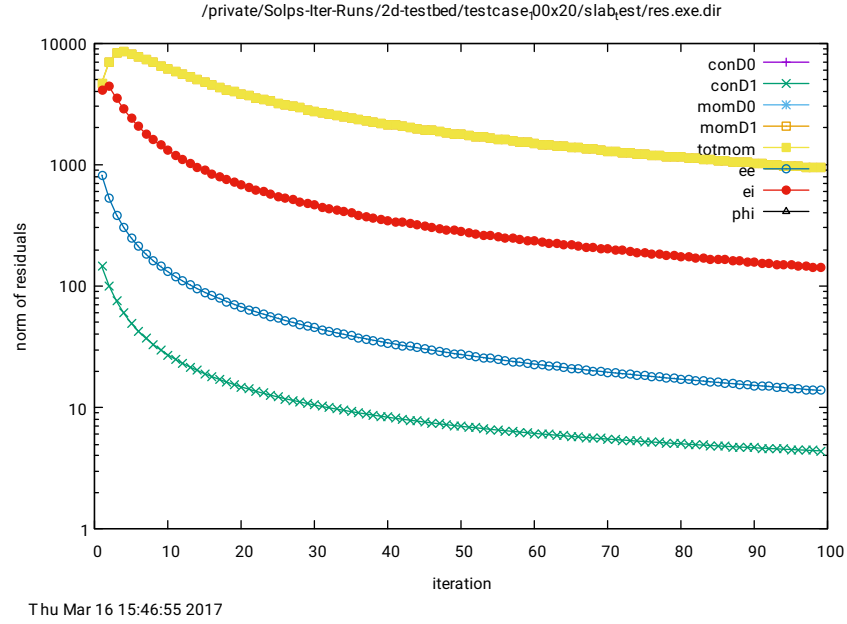


Figure 5: Residuals of the running averaged solution for a case where the averaging was started too early

Acknowledgement

This work has been carried out within the framework of the EUROfusion Consortium and has received funding from the Euratom research and training programme 2014–2018 under grant agreement No 633053. The views and opinions expressed herein do not necessarily reflect those of the European Commission. Parts of the work are supported by the Research Foundation Flanders (FWO) under project grant G078316N. The work of K. Ghooos is sponsored by the Institute for the Promotion of Innovation through Science and Technology in Flanders (IWT- Vlaanderen).

References

- [1] K. Ghooos, W. Dekeyser, G. Samaey, P. Börner, D. Reiter, and M. Baelmans, “Accuracy and convergence of coupled finite-volume / Monte-Carlo codes for plasma edge simulations of nuclear fusion reactors”, *Contributions to Plasma Physics*, vol. 56, no. 6-8, pp. 616–621, 2016. Plasma Edge Theory Conference, Nara, Japan.
- [2] K. Ghooos, W. Dekeyser, G. Samaey, and M. Baelmans, “Accuracy and convergence of coupled finite-volume / Monte-Carlo codes for plasma edge simulations of nuclear fusion reactors”, *Journal of Computational Physics*, vol. 322, pp. 162–182, 2016. 2nd Frontiers in Computational Physics: Energy conference, Zurich, Switzerland.
- [3] M. Baelmans, P. Börner, K. Ghooos, and G. Samaey, “Efficient code simulation strategies for B2-EIRENE”, *Nuclear Materials and Energy*, vol. 1-6, 2016.
- [4] K. Ghooos, P. Börner, W. Dekeyser, A. Kukushkin, and M. Baelmans, “Grid resolution study for B2-EIRENE simulation of partially detached ITER divertor plasma”, *Nuclear Fusion*, vol. 59, p. 026001, 2019.
- [5] K. Ghooos, G. Samaey, and M. Baelmans, “Numerical error estimation in Random Noise coupled plasma edge simulations in nuclear fusion reactors”, *M&C 2017, Jeju, Korea, April 16-20, 2017*.

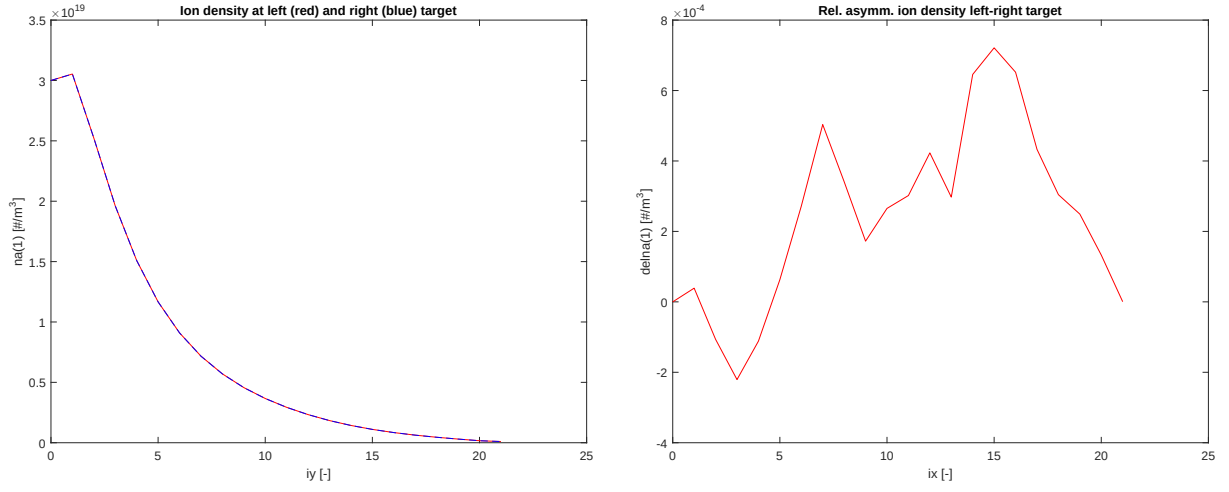


Figure 6: Ion density profile (left) and its relative asymmetry (right) for the slab test case

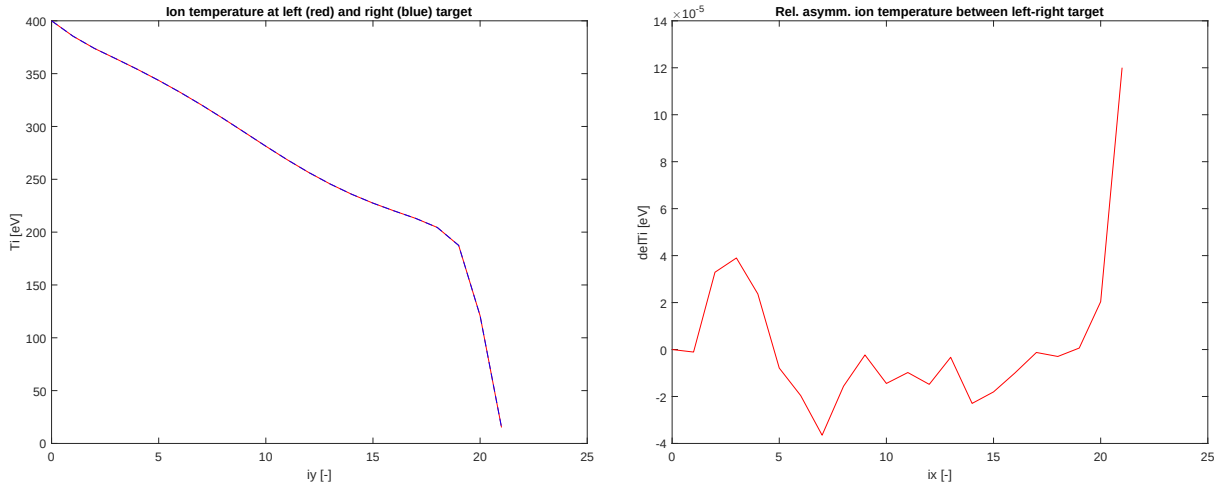


Figure 7: Ion temperature profile (left) and its relative asymmetry (right) for the slab test case

Variable	Averaged value	Relative Statistical error
nese μ _av	$1.84 \cdot 10^{18}$	0.023%
nese π _av	$2.63 \cdot 10^{18}$	0.073%
nese ρ _av	$2.63 \cdot 10^{18}$	0.055%
tesep μ _av	38.8	0.024%
tesep π _av	33.6	0.041%
tesep ρ _av	33.6	0.043%
tisep μ _av	264	0.031%
tisep π _av	263	0.031%
tisep ρ _av	263	0.031%
posep μ _av	120	0.024%
posep π _av	104	0.041%
posep ρ _av	104	0.043%

Table 5: Statistical errors for averaged quantities in slab geometry after 100 batches of 500 iterations (note: maximum values are not shown as they occur at the preset core boundary for this case)

Variable	Value	Description
b2mndt_av	(Def. 0) 0 1	Controls computation of running average No computation of running average Computation of running average
b2mndt_av_continue	(Def. 1) 0 1	Controls continuation of running average Restart of running averages is explicitly set Running averages start from data previously stored in b2faveri . If data is not available, averaging is restarted
b2mndt_av_ntim_run	(Def. 1000) 0 > 0	Number of it. for writing running average No writing of running average Number of iterations for writing running average
b2mndt_av_ntim_batch	(Def. 500) 0 > 0	Number of iterations used for batch averages No batch averaging Number of iterations used for batch average
b2mndt_av_batch_all	(Def. 0) 0 1	Controls output quantities for batch averages Default set output with limited monitoring points Entire 2D fields of batch averages and variances for n_e , T_e , ... are written to directory <code>./batch_av/batch_av.XXXX</code> . Expert use only! – requires lot of storage!
b2mndr_av_read	(Def. 0) 0 1	Controls initial plasma (instantaneous or averaged) Initial plasma is instantaneous plasma (from b2fstati) Initial plasma is based on averaged state (from b2faveri)

Table 6: New variables introduced for averaging and error assessment